Algoritmos y Estructuras de Datos

Guía Práctica 8
Sorting
Segundo Cuatrimestre 2026

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ejercicio 1. Comparar la complejidad de los algoritmos de ordenamiento dados en la teórica para el caso en que el arreglo a ordenar se encuentre perfectamente ordenado de manera *inversa* a la deseada.

Solución

Partiendo de la premisa de que el arreglo a ordenar se encuentre perfectamente ordenado de manera inversa a la deseada, se tienen las siguientes complejidades:

- SelectionSort: $O(n^2)$
- InsertionSort: $O(n^2)$
- QuickSort: Depende de la elección del pivote, si se elige al azar tiene peor caso $O(n^2)$, y caso promedio $O(n \log n)$. En caso de que se elija como pivote el elemento del medio, la complejidad pasa a ser $O(n \log n)$
- MergeSort: $O(n \log n)$
- HeapSort: $O(n \log n)$

Ejercicio 2. Defina la propiedad de estabilidad en un algoritmo de ordenamiento. Explique por qué qué el algoritmo de *heapSort* no es estable.

Solución

Un algoritmo de ordenamiento es estable si no modifica el orden relativo de los elementos que tienen igual valor según nuestro criterio de ordenamiento.

HeapSort no es estable porque, en el primer paso, el algoritmo de heapify rompe el orden relativo de los elementos iguales al realizar las subidas y bajas en la representación de Heap sobre array. Además, a la hora de desencolar los elementos e ir agregándolos atrás de nuestro array, puede ocurrir que no se mantenga dicho orden, como por ejemplo en el caso de [2prim, 2seg, 1] $\rightarrow$ [2seg, 1, 2prim] $\rightarrow$ [1, 2seg, 2prim], quedando el primer 2 en una posición diferente relativa a la original

Ejercicio 3. Imagine secuencias de naturales de la forma $s = Concatenar(s', s'')$, donde s' es una secuencia ordenada de naturales y s'' es una secuencia de naturales elegidos al azar. ¿Qué método utilizaría para ordenar s? Justificar. (Entiéndase que s' se encuentra ordenada de la manera deseada.)

Solución

Para ordenar la secuencia s, podemos seguir los siguientes pasos:

- **Encontrar el punto donde termina s' y comienza s''**:

  Podemos hacer esto iterando sobre la secuencia s y comparando cada elemento con el siguiente, cuando encontramos un elemento que es mayor que el siguiente, sabemos que encontramos el punto de separación. Este proceso tiene una complejidad temporal de $O(n)$, donde n es la longitud total de la secuencia s.

- **Ordenar la subsecuencia s''**: Aplicar un algoritmo de ordenamiento, para ordenar la subsecuencia s'' (por ej. MergeSort). La complejidad de este paso es $O(m \log m)$, donde m es la longitud de s''. En el peor de los casos, $m = n$, lo que resulta en una complejidad de $O(n \log n)$.

- **Mergear s' y s'' ordenados**: Una vez que s'' está ordenada, mergearla con s' utilizando el algoritmo *Merge* de *MergeSort*. Complejidad de Merge: $O(|s'| + |s''|) = O(n)$.

Pseudocódigo:

```
proc ordenarConcat(s: array<int>)
    vectorOrdenado := new Vector<int>
    vectorDesordenado := new Vector<int>
    vectorOrdenado.AgregarAtras(s[0])

    flagEstoyEnOrdenado := True
    for(int i := 1; i < s.length; i++): // O(n)
        if(s[i] > s[i-1] and flagEstoyEnOrdenado):
            flagEstoyEnOrdenado := false

        if(flagEstoyEnOrdenado):
            vectorOrdenado.agregarAtras(s[i]) // O(1) amortizado
        else:
            vectorDesordenado.agregarAtras(s[i]) // O(1) amortizado
    endfor

    mergeSort(vectorDesordenado, deMenorAMayor) // O(nlog(n))

    s := Merge(vectorOrdenado, vectorDesordenado) // O(n)
endproc
```

Complejidad Total: $O(n \log n)$

Implementación del algoritmo de Merge que utiliza MergeSort:

```
//Lo escribo con vectores, pero puede recibir arrays también
// l = |v1| y m = |v2|
proc Merge(v1: Vector<int>, v2: Vector<int>) : Vector<int>
    res := new Vector<int>
    i := 0
    j := 0

    // Recorro ambos vectores aprovechando que están ordenados y me quedo
    //con los elementos menores
    while(i < v1.longitud() and j < v2.longitud()): // O(l + m)
        if(v1[i] <= v2[j]):
            res.agregarAtras(v1[i]) // O(1) amortizado
            i++
        else:
            res.agregarAtras(v2[j]) // O(1) amortizado
            j++
    endwhile

    // Agregar los elementos restantes de v1 (si los hay)
    while(i < v1.longitud()): // O(l)
        res.agregarAtras(v1[i]) // O(1) amortizado
        i++
    endwhile

    // Agregar los elementos restantes de v2 (si los hay)
    while(j < v2.longitud()): // O(m)
        res.agregarAtras(v2[j]) // O(1) amortizado
        j++
    endwhile

    return res
endproc
```

Complejidad de Merge: $O(l + m)$

Ejercicio 4. Escribir un algoritmo que encuentre los k elementos más chicos de un arreglo de dimensión n, donde $k \leq n$.

¿Cuál es su complejidad temporal? ¿A partir de qué valor de k es ventajoso ordenar el arreglo entero primero?

Solución

Para encontrar los k elementos más pequeños de un arreglo, podemos considerar los siguientes métodos:

a) **Iterar k veces sobre la lista:**

- **Descripción:** En cada iteración, recorrer todo el arreglo para encontrar el mínimo y seleccionarlo.
- **Complejidad Temporal:** $O(k \cdot n)$

b) **Ordenar el arreglo primero y luego seleccionar los k elementos más pequeños:**

- **Descripción:** Utilizar un algoritmo de ordenamiento para ordenar el arreglo completo y luego tomar los primeros k elementos.
- **Complejidad Temporal:** $O(n \log n) + O(k) = O(n \log n)$
- **Ventaja:** Este método es más eficiente que el anterior cuando $\log n < k$.

Pseudocódigo de la opción b:

```
proc encontrarKMasChicos(a: array<int>, k: int): array<int>
    mergeSort(a, menorAMayor) // O(nlog(n))
    res := new array<int>(k)

    for(int i := 0; i< k; i++): // O(k)
        res[i] := a[i]
    endfor

    return res
endproc
```

Ejercicio 5. Se tiene un conjunto de n secuencias $\{s_1, s_2, \ldots, s_n\}$ en donde cada s_i $(1 \leq i \leq n)$ es una secuencia ordenada de naturales. ¿Qué método utilizaría para obtener un arreglo que contenga todos los elementos de la unión de los s_i ordenados. Describirlo. Justificar.

Solución

- **Opción 1:** Podemos utilizar la misma estrategia que utiliza MergeSort para combinar dos secuencias ordenadas, pero en este caso, lo extendemos para combinar n secuencias ordenadas. El algoritmo sería igual al Merge de MergeSort, pero mantendría n índices, uno por cada secuencia, y en cada paso, se selecciona el menor de los elementos apuntados por los índices y se avanza el índice correspondiente. La complejidad de este algoritmo es $O(n \cdot m)$, donde m es la longitud total de todas las secuencias (m = $\sum_{i=1}^{n} |s_i|$).
- **Opción 2:** Otra opción es concatenar todas las secuencias en un solo arreglo y luego ordenarlo con un algoritmo de ordenamiento eficiente, como MergeSort o HeapSort. La complejidad de esta opción es $O(m \log m)$, es más eficiente que la anterior cuando $\log m < n$.
- **Opción 3:** Hacer una estrategia similar a la opción 1, es decir, utilizar la estrategia de Merge, pero, para saber cual elemento de las listas es el más chico de todos, mantener un minHeap el cual siempre tendrá tamaño n. Dicho Heap tendrá tuplas<elem, indice de la secuencia donde provino, indice interno de la secuencia donde provino>, por lo que, una vez desencolado del heap y puesto en el array solución, puedo encolar el próximo elemento de dicha lista en caso de que no haya usado todos ya. Esto toma O(log(n)*m), logrando la mejor complejidad de las tres

Pseudocódigo de la opción 2:

```
proc ordenarNSecuencias(nSeq: array<array<int>>)
    vectorUnido := new Vector<int>

    for(int i := 0; i < nSeq.length; i++):  // O(m)
        for(int j := 0; j < nSeq[i].length; j++):
            vectorUnido.agregarAtras(nSeq[i][j]) // O(1) amortizado
```

```
            endfor
        endfor

        mergeSort(vectorUnido, deMenorAMayor) // O(mlog(m))

        res := vectorToArray(vectorUnido) // funcion trivial en O(m)

        return res
    endproc
```

Ejercicio 6. Se tiene un arreglo de n números naturales que se quiere ordenar por frecuencia, y en caso de igual frecuencia, por su valor. Por ejemplo, a partir del arreglo [1, 3, 1, 7, 2, 7, 1, 7, 3] se quiere obtener [1, 1, 1, 7, 7, 7, 3, 3, 2]. Describa un algoritmo que realice el ordenamiento descripto, utilizando las estructuras de datos intermedias que considere necesarias. Calcule el orden de complejidad temporal del algoritmo propuesto.

Solución

a) **Ordenamos por valor:**

- Utilizamos un algoritmo de ordenamiento para ordenar los elementos del arreglo.
- Complejidad: $O(n \log n)$.

b) **Creamos vector auxiliar con frecuencias:**

- Construimos un vector de tuplas $(valor, frecuencia)$ que contenga los valores y sus frecuencias.
- Para ello, recorremos el arreglo ordenado y contamos las frecuencias de cada valor, cuando el valor cambia, guardamos el valor anterior y su frecuencia en el vector auxiliar.
- Complejidad: $O(n)$.

c) **Ordenamos el vector auxiliar por frecuencia:**

- Utilizamos un algoritmo de ordenamiento **estable** (por ej. MergeSort) para ordenar el vector auxiliar por frecuencia.
- Es importante que el algoritmo de ordenamiento sea estable para que en caso de igual frecuencia, se mantenga el orden del primer ordenamiento por valor.
- Complejidad: $O(n \log n)$.

d) **Construimos el arreglo resultado:**

- Recorremos el vector auxiliar ordenado y creamos el arreglo resultado.
- Para cada tupla $(valor, frecuencia)$, agregamos $valor$ $frecuencia$ veces el valor al arreglo resultado.
- Complejidad: $O(n)$.

Complejidad Total: $O(n \log n)$

Pseudocódigo:

```
proc ordenarFrecuencia(arrayF: array<int>)
    mergeSort(arrayF, deMenorAMayor) // O(nlog(n))
    vectorAux := new vector<tuple<int, int>>
    vectorAux.agregarAtras(<arrayF[0], 1>)

    indiceVector := 0

    for(int i := 1; i< arrayF.length; i++): // O(n)

        if(arrayF[i] != vectorAux[indiceVector][0]):
            vectorAux.agregarAtras(<arrayF[i], 1>) //O(1) amortizado
            indiceVector++
```

```
            else:
                vectorAux[indiceVector][1] := vectorAux[indiceVector][1] + 1
        endfor

        mergeSort(vectorAux, mayorAMenorPorSegundoComponenteDeTupla) // O(nlog(n))

        indiceRes := 0
        for(int i := 0; i< vectorAux.longitud(); i++): // O(n)

            for(int j := 0; j< vectorAux[i][1]; j++):
                arrayF[indiceRes] := vectorAux[i][0]
                indiceRes++
            endfor

        endfor

    endproc
```

Ejercicio 7. Sea $A[1 \dots n]$ un arreglo que contiene n números naturales. Diremos que un rango de posiciones $[i \dots j]$, con $1 \leq i \leq j \leq n$, contiene una escalera en A si valen las siguientes dos propiedades:

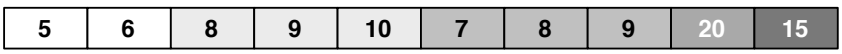
a) $(\forall k : \mathbb{N})(i \leq k < j) \implies (A[k+1] = A[k] + 1)$ (esto es, los elementos no sólo están ordenados en forma creciente, sino que además el siguiente vale exactamente uno más que el anterior).

b) Si $1 < i$ entonces $A[i] \neq A[i-1] + 1$ y si $j < n$ entonces $A[j+1] \neq A[j] + 1$ (la propiedad es *maximal*, es decir que el rango no puede extenderse sin que deje de ser una escalera según el punto anterior).

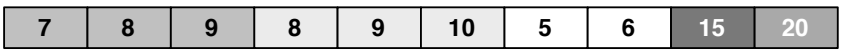
Se puede verificar fácilmente que cualquier arreglo puede ser descompuesto de manera única como una secuencia de escaleras. Se pide escribir un algoritmo para reposicionar las escaleras del arreglo original, de modo que las mismas se presenten en orden decreciente de longitud y, para las de la misma longitud, se presenten ordenadas en forma creciente por el primer valor de la escalera.

El resultado debe ser del mismo tipo de datos que el arreglo original. Calcule la complejidad temporal de la solución propuesta, y justifique dicho cálculo.

Por ejemplo, el siguiente arreglo



debería ser transformado a



Ayuda: se aconseja comenzar el ejercicio con una clara descripción en castellano de la estrategia que propone para resolver el problema.

Solución

Para resolver este problema, podemos hacer algo similar al ejercicio anterior, pero en lugar de ordenar por frecuencia, ordenamos por longitud de la escalera y en caso de igual longitud, por el primer valor de la escalera.

a) **Creamos un vector auxiliar con la info de las escaleras:**

- Recorremos el arreglo y encontramos todas las escaleras.
- Para ello, recorremos el arreglo y cuando encontramos un valor que no es igual al anterior más uno, guardamos el inicio de la escalera.
- Para cada escalera, guardamos el inicio y longitud.
- Complejidad: $O(n)$.

b) **Ordenar las escaleras:**

- Ordenamos las escaleras por longitud (decreciente) y en caso de igual longitud, por el primer valor de la escalera.
- Obs: en vez de ordenar dos veces usando un algoritmo estable, lo cual también sería correcto, podríamos definir un criterio de ordenamiento que tenga en cuenta dos condiciones en vez de solo una.
- Complejidad: $O(m \log m)$, donde m es la cantidad de escaleras.

c) **Creamos arreglo resultado:**

- Recorremos las escaleras ordenadas y construimos el arreglo resultado.
- Complejidad: $O(n)$.

Complejidad Total: $O(n + m \log m)$, en peor caso: $O(n \log n)$.

Pseudocódigo:

```
proc ordenarEscaleras(escaleras: array<int>)
    vectorAux := new vector<tuple<int, int>> //<primer valor escalera, largo>
    vectorAux.agregarAtras(<escaleras[0], 1>)

    indiceVector := 0

    for(int i := 1; i< escaleras.length; i++): // O(n)

        if(escaleras[i] != escaleras[i-1]):
            vectorAux.agregarAtras(<escaleras, 1>) //O(1) amortizado
            indiceVector++

        else:
            vectorAux[indiceVector][1] := vectorAux[indiceVector][1] + 1
    endfor

    mergeSort(vectorAux, MenorAMayorPorPrimerComponenteDeTupla) // O(mlog(m))

    mergeSort(vectorAux, mayorAMenorPorSegundoComponenteDeTupla) // O(mlog(m))

    indiceRes := 0
    for(int i := 0; i< vectorAux.longitud(); i++): // O(n)

        for(int j := 0; j< vectorAux[i][1]; j++):
            escaleras[indiceRes] := vectorAux[i][0] + j
            indiceRes++
        endfor

    endfor

endproc
```

Ejercicio 8. Suponga que su objetivo es ordenar arreglos de naturales (sobre los que no se conoce nada en especial), y que cuenta con una implementación de árboles AVL. ¿Se le ocurre alguna forma de aprovecharla? Conciba un algoritmo que llamaremos *AVL Sort*. ¿Cuál es el mejor orden de complejidad que puede lograr?

Ayuda: debería hallar un algoritmo que requiera tiempo $O(n \log d)$ en peor caso, donde n es la cantidad de elementos a ordenar y d es la cantidad de elementos distintos. Para inspirarse, piense en *Heap Sort* (no en los detalles puntuales, sino en la idea general de este último).

Justifique por qué su algoritmo cumple con lo pedido.

Solución

a) **Construcción del AVL:**

- Insertamos todos los elementos del arreglo en el *AVL*, junto con un valor extra para guardar las repeticiones. O sea, en lugar de insertar el elemento directamente, insertamos una tupla (elemento, repeticiones). Vale aclarar, que este AVL va a estar ordenado por el valor de los elementos.
- Vamos a iterar sobre todo el arreglo (n iteraciones):
  - En cada iteración, chequeamos si el elemento actual ya está en el AVL en $O(\log d)$, si no está, insertamos la tupla (elemento, 1) en $O(\log d)$. Si ya estaba, incrementamos las repeticiones en 1.
- Complejidad: $O(n \log d)$.

b) **Recorrido en orden del AVL para construir arreglo ordenado:**

- Podemos utilizar el algoritmo inorder para recorrer el AVL en orden y construir un arreglo ordenado.
- En cada nodo, agregamos el elemento tantas veces como repeticiones tenga.
- Complejidad: $O(n)$.

Complejidad Total: $O(n \log d)$.

Pseudocódigo:

```
proc AVLSort(array: array<int>)
    dictAux := new DiccionarioLog<int, int>

    for elem in array: // O(nlog(d))
        if(dictAux.pertenece(elem)):
            dictAux.definir(elem, dictAux.obtener(elem)+1)
        else:
            dictAux.definir(elem, 1)
    endfor

    indiceRes := 0
    for <clave, valor> in dictAux: // O(n)
        for(int i := 0; i<valor; i++):
            array[indiceRes] := clave
            indiceRes++
endproc
```

Ejercicio 9. Se tienen dos arreglos de números naturales, $A[1..n]$ y $B[1..m]$. Nada en especial se sabe de B, pero A tiene n' secuencias de números repetidos continuos (por ejemplo $A = [333331111888887777145555], n' = 7$). Se sabe además que n' es mucho más chico que n. Se desea obtener un arreglo C de tamaño $n+m$ que contenga los elementos de A y B, ordenados.

a) Escriba un algoritmo para obtener C que tenga complejidad temporal $O(n + (n' + m)\log(n' + m))$ en el peor caso. Justifique la complejidad de su algoritmo.

b) Suponiendo que todos los elementos de B se encuentran en A, escriba un algoritmo para obtener C que tenga complejidad temporal $O(n + n'(\log(n') + m))$ en el peor caso y que utilice solamente arreglos o vectores como estructuras auxiliares. Justifique la complejidad de su algoritmo.

Solución

a) **Algoritmo 1:**

a) **Construcción del vector auxiliar para A:**
- Construimos un vector auxiliar de tuplas (valor, repeticiones) con los elementos de A.
- Para ello, recorremos el arreglo y creamos un vector para ir agregando los elementos
- **Complejidad**: $O(n)$.

b) **Construcción del arreglo vector para B:**
- Construimos un vector auxiliar con los elementos de B para poder concatenarlo con el vector auxiliar de A.

- Este paso se realiza únicamente para simplificar la solución. En este caso, no es necesario contar las repeticiones de los elementos de B; simplemente guardamos tuplas $(\text{valor}, 1)$.
- **Complejidad**: $O(m)$.

c) **Ordenamiento del vector auxiliar:**

- Concatenamos los vectores auxiliares de A y B.
- Ordenamos el vector resultante por el valor del primer componente de las tuplas.
- **Complejidad**: $O\big((n' + m)\log(n' + m)\big)$.

d) **Construcción del arreglo resultado:**

- Recorremos el vector ordenado y reconstruimos un arreglo final ordenado.
- **Complejidad**: $O(n + m)$.

Complejidad Total: $O\big(n + (n' + m)\log(n' + m)\big)$.

b) **Algoritmo 2:**

a) **Construcción del vector auxiliar para A:**

- Repetimos el primer paso del inciso anterior para construir el vector auxiliar de A.

b) **Agregación de repeticiones de elementos de B al vector auxiliar de A:**

- Para cada elemento de B, buscamos en el vector auxiliar de A. Si lo encontramos, incrementamos la cantidad de repeticiones.
- **Complejidad**: $O(n' \cdot m)$.

c) **Ordenamiento del vector auxiliar:**

- Ordenamos el vector auxiliar por el valor de las tuplas.
- **Complejidad**: $O(n' \log n')$.

d) **Construcción del arreglo resultado:**

- Recorremos el vector ordenado y reconstruimos un arreglo final ordenado, similar al inciso anterior.
- **Complejidad**: $O(n + m)$.

Complejidad Total: $O\big(n + n'(\log n' + m)\big)$.

Pseudocódigo A:

```
proc ordenarArreglosA(A: array<int>, B: A: array<int>): array<int>
    vectorAuxA := new Vector<tuple<int, int>>
    vectorAuxA.agregarAtras(<A[0], 1>)

    indiceVector := 0
    for(int i := 1; i< A.length; i++): // O(n)
        if(A[i] != vectorAuxA[indiceVector][0]):
            vectorAuxA.agregarAtras(<A[i], 1>) // O(1) amortizado
            indiceVector++
        else:
            vectorAuxA[indiceVector][1] := vectorAuxA[indiceVector][1] + 1
    endfor

    vectorAuxB := new Vector<tuple<int, int>>

    for(int i := 0; i< B.length; i++): // O(m)
        vectorAuxB.agregarAtras(<B[i], 1>) // O(1) amortizado
    endfor

    vectorConcat := vectorAuxA.concat(vectorAuxB) // O(m)

    mergeSort(vectorConcat, menorAMayorPorPrimerComponente) // O((n'+m)log(n'+m))

    CRes := new array<int>[A.length+B.length]

    indiceC := 0
    for elem in vectorConcat: // O(n+m)
```

```
        for(int i:= 0; i< elem[1];i++):
            CRes[indiceC] := elem[0]
            indiceC++

    return CRes

endproc
```

Pseudocódigo B:

```
proc ordenarArreglosB(A: array<int>, B: A: array<int>): array<int>
    vectorAux := new Vector<tuple<int, int>>
    vectorAux.agregarAtras(<A[0], 1>)

    indiceVector := 0
    for(int i := 1; i< A.length; i++): // O(n)
        if(A[i] != vectorAux[indiceVector][0]):
            vectorAux.agregarAtras(<A[i], 1>) // O(1) amortizado
            indiceVector++
        else:
            vectorAux[indiceVector][1] := vectorAux[indiceVector][1] + 1
    endfor

    for(int i := 0; i< B.length; i++): // O(m*n')
        encontrado := false
        j := 0
        while(not encontrado):
            if(vectorAux[j][0] == B[i]):
                vectorAux[j][1] := vectorAux[j][1] + 1
                encontrado := true

            j := j + 1

    endfor

    mergeSort(vectorAux, menorAMayorPorPrimerComponente) // O(n'log(n'))

    CRes := new array<int>[A.length+B.length]

    indiceC := 0
    for elem in vectorAux: // O(n+m)
        for(int i:= 0; i< elem[1];i++):
            CRes[indiceC] := elem[0]
            indiceC++

    return CRes

endproc
```

Ejercicio 10. Considere la siguiente estructura para guardar las notas de un alumno de un curso:

Turno ES enum{Mañana, Noche}
Alumno ES struct $\langle nombre : \text{string}, \ turno : Turno, \ puntaje : \mathbb{Z} \rangle$
donde *puntaje* es un número entre 0 y no mayor que 10.

Se necesita ordenar un arreglo de alumnos de forma tal que todes les alumnes de la mañana aparezcan al inicio de la tabla según un orden creciente de notas y todos los de la noche aparezcan al final de la tabla también ordenados de manera creciente respecto de su puntaje, como muestra en el siguiente ejemplo:

Entrada

Ana	*Mañana*	10
Rita	*Noche*	6
Juan	*Mañana*	6
Paula	*Mañana*	7
Jose	*Noche*	7
Pedro	*Noche*	8

Salida

Juan	*Mañana*	6
Paula	*Mañana*	7
Ana	*Mañana*	10
Rita	*Noche*	6
Jose	*Noche*	7
Pedro	*Noche*	8

a) Proponer un algoritmo de ordenamiento `proc ordenaPlanilla`(inout p : Array$\langle alumno \rangle$) para resolver el problema descripto anteriormente y cuya complejidad temporal sea $O(n)$ en el peor caso, donde n es la cantidad de elementos del arreglo. Justificar.

b) Modificar la solución del inciso anterior para funcionar en el caso que Turno sea un tipo enumerado con más elementos (donde la cantidad de los mismos sigue estando acotada y el orden final está dado por el orden de los valores en el enum. Puedo hacer **for** t **in** Turno).

c) ¿La cota $O(n)$ contradice el "lower bound" sobre la complejidad temporal en el peor caso de los algoritmos de ordenamiento? (El Teorema de "lower bound" establece que todo algoritmo general de ordenamiento tiene complejidad temporal $\Omega(n \log n)$.) Explique su respuesta.

Solución

a)

- **Algoritmo a**
  - Ordenamos por nota (campo puntaje del struct) usando bucketSort con 11 categorias en O(n+11) = O(n)
  - Ordenamos por turno (campo turno del struct) usando bucketSort (observar que es estable) con 2 categorias en O(n+2) = O(n)
  - **Complejidad**: $O(n)$.

b)

- **Algoritmo b**
  - Ordenamos por nota (campo puntaje del struct) usando bucketSort con 11 categorias en O(n+11) = O(n)
  - Ordenamos por turno (campo turno del struct) usando bucketSort (observar que es estable) con t categorias, siendo t la cantidad de turnos del enum Turno en O(n+t) con t una constante entonces O(n)
  - **Complejidad**: $O(n)$.

c) No lo contradice porque dicho teorema habla de los algoritmos de ordenamiento por comparaciones, y en este algoritmo no estamos comparando los elementos entre sí para ordenar, sino que nos aprovechamos que tenemos cierta información de antemano para poder aplicar otras estrategias de ordenamiento en tiempo lineal

Pseudocódigo A:

```
proc ordenarAlumnosA(A: array<struct<nombre:string, turno:Turno, puntaje:int>>)

    bucketSort(A, 11, h)
    //Con h la cual asigna los elementos del arreglo según el campo puntaje
    //puntaje 0 al bucket 0, puntaje 1 al bucket 1, ..., puntaje 10 al bucket 10
    //Claramente h se ejecuta en O(1)
    //O(n+11) = O(n)

    bucketSort(A, 2, h')
    //Con h' la cual asigna los elementos del arreglo según el campo turno
    //turno Mañana va al bucket 0 y turno Noche al bucket 1
    //Claramente h' se ejecuta en O(1)
    //O(n+2) = O(n)

endproc
```

Pseudocódigo B:

```
proc ordenarAlumnosB(A: array<struct<nombre:string, turno:Turno, puntaje:int>>)

    bucketSort(A, 11, h)
    //Con h la cual asigna los elementos del arreglo según el campo puntaje
    //puntaje 0 al bucket 0, puntaje 1 al bucket 1, ..., puntaje 10 al bucket 10
    //Claramente h se ejecuta en O(1)
    //O(n+11) = O(n)


    t = len(Turno) //Asumo que tengo el Enum Turno como si fuera un arreglo
    bucketSort(A, t, h')
    //Con h la cual asigna los elementos del arreglo según el campo turno
    //Turno[0] va al bucket 0, Turno[1] al bucket 1, Turno[t-1] va al bucket t-1
    //Claramente h' se ejecuta en O(1)
    //O(n+t) = O(n)

endproc
```

Ejercicio 11. Sea $A[1 \dots n]$ un arreglo de números naturales *en rango* (cada elemento está en el rango de 1 a k, siendo k alguna constante). Diseñe un algoritmo que ordene esta clase de arreglos en tiempo $O(n)$. Demuestre que la cota temporal es correcta.

Solución

Para resolver este ejercicio podemos utilizar Counting Sort adaptado para valores dentro de un intervalo, en este caso de 1 a k.

La complejidad de Counting Sort es $O(n + (k-1))$, pero como $k-1$ es una constante, la complejidad se simplifica a $O(n)$.

```
proc ordenarEnIntervalo(Array: array<int>)
    countingSort(Array, 1, k) // O(n + (k-1)) = O(n)
endproc
```

Ejercicio 12. Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un número entero positivo. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, a lo sumo $\lfloor \sqrt{n} \rfloor$ valores están fuera del rango $[20, 40]$.

Proponer un algoritmo $O(n)$ que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

Solución

Podemos definir nuestra estrategia de la siguiente manera:

a) **Separamos el arreglo original en dos: uno con los valores en el rango [20, 40] y otro con los valores fuera del rango:**

- Recorremos el arreglo y separamos los valores en dos arreglos: uno con los valores en el rango [20, 40] y otro con los valores fuera del rango.
- Para hacer esto podemos ayudarnos con dos listas enlazadas.
- **Complejidad**: $O(n)$.

b) **Ordenamos los valores en el rango [20, 40]:**

- Utilizamos Counting Sort para ordenar los valores en el rango [20, 40].
- La complejidad de Counting Sort es $O(n)$.

c) **Ordenamos los valores fuera del rango [20, 40]:**

- Utilizamos Selection Sort para ordenar los valores fuera del rango [20, 40].
- La complejidad de Selection Sort es cuadrática en el peor caso, pero como a lo sumo $\lfloor\sqrt{n}\rfloor$ valores están fuera del rango, la complejidad se simplifica a $O(\sqrt{n}\sqrt{n})$.
- **Complejidad**: $O(n)$.

d) **Combinamos los dos arreglos ordenados:**

- Para hacer esto, podemos utilizar el algoritmo Merge de MergeSort. Como la suma de los tamaños de los dos arreglos es n, la complejidad de Merge es $O(n)$.
- **Complejidad**: $O(n)$.

Pseudocódigo: Resuelto en el campus clase Sorting

Ejercicio 13. Se tiene un arreglo $A[1 \dots n]$ de T, donde T son tuplas $\langle c_1 : \mathbb{N} \times c_2 : string[\ell]\rangle$ y los $string[\ell]$ son *strings* de longitud máxima ℓ. Si bien la comparación de dos $\mathbb{N}$ toma $O(1)$, la comparación de dos $string[\ell]$ toma $O(\ell)$. Se desea ordenar A en base a la primer componente en orden ascendente y, en caso de empate, por la segunda en orden lexicográfico.

a) Escriba un algoritmo que tenga complejidad temporal $O(n\ell + n\log(n))$ en el peor caso. Justifique la complejidad de su algoritmo.

b) Suponiendo que los naturales de la primer componente están acotados, adapte su algoritmo para que tenga complejidad temporal $O(n\ell)$ en el peor caso. Justifique la complejidad de su algoritmo.

Solución

a)

- **Algoritmo a**
  - Ordeno según segunda componente en orden lexicográfico usando radixSort en O(n*l)
  - Ordeno según primer componente de forma ascendente usando mergeSort que es estable en O(n*log(n))
  - **Complejidad**: $O(n * l + n * log(n))$.

b)

- **Algoritmo b**
  - Ordeno según segunda componente en orden lexicográfico usando radixSort en O(n*l)
  - Ordeno con bucketSort, el cual es estable. Va a tener como categorias a los números que están en dicho intervalo acotado, por lo que las categorias son acotadas y la función de asignación h, en este caso, es constante. O(n+tamaño acotado del intervalo) = O(n)
  - **Complejidad**: $O(n * l)$.

Pseudocódigo A:

```
proc ordenarTuplasA(A: array<tuple<int, string[l]>>)

    radixSort(A, l)
    //En orden lexicográfico por segundo componente. //O(n*l)

    mergeSort(A, AscendentePorPrimerComponente) //Estable O(n*log(n))

endproc
```

Pseudocódigo B:

```
proc ordenarTuplasB(A: array<tuple<int, string[l]>>)

    radixSort(A, l)
    //En orden lexicográfico por segundo componente. //O(n*l)
```

```
        //Asumo que tengo definidas las constantes min y max las cuales vienen dadas
        //Debido a que el rango del 1er componente de los elementos de A están acotados

        bucketSort(A, max–min+1, h)
        //h asigna los elementos cuyo primer elemento de la tupla es min al bucket 0
        //min+1 al bucket 1, ..., max al bucket (max–min)
        //La función h es constante, es similar a como lo hace CountingSort
        //O(n+(max–min)), pero max y min son constantes, entonces es O(n) y es estable

    endproc
```

Ejercicio 14. Se tiene un arreglo A de n números naturales y un entero k. Se desea obtener un arreglo B ordenado de $n \times k$ naturales que contenga los elementos de A multiplicados por cada entero entre 1 y k, es decir, para cada $1 \leq i \leq n$ y $1 \leq j \leq k$ se debe incluir en la salida el elemento $j \times A[i]$. Notar que podría haber repeticiones en la entrada y en la salida.

a) Implementar la función

$$\texttt{proc ordenarMúltiplos}(\text{in } A : \text{Array}\langle\mathbb{N}\rangle, \text{in } k : \mathbb{N}) : \text{Array}\langle\mathbb{N}\rangle$$

que resuelve el problema planteado. La función debe ser de tiempo $O(nk \log n)$, dónde n es el tamaño del arreglo.

b) Calcular y justificar la complejidad del algoritmo propuesto.

Solución

a) **Construcción del vector auxiliar:**

- Construimos un vector auxiliar para mantener la información de los elementos de A.
- En principio, guardamos elementos con los siguientes campos: valorOriginal, valorActual, multiploActual.
- Por ejemplo para el arreglo $A = [3, 1, 4]$ y $k = 3$, el vector auxiliar sería: $[(3, 3, 1), (1, 1, 1), (4, 4, 1)]$.
- **Complejidad**: $O(n)$.

b) **Creamos un Heap con los elementos del vector auxiliar:**

- Creamos un MinHeap con los elementos del vector auxiliar, ordenado por el valorActual, utilizando Floyd.
- **Complejidad**: $O(n)$.

c) **Ordenamos:**

- Creamos un arreglo de tamaño n*k para guardar el resultado.
- Mientras el heap no esté vacío, extraemos el mínimo y agregamos su valorActual al arreglo
- Luego, si el multiploActual es menor que k, agregamos una nueva tupla al heap con el
  - valorOriginal:= valorOriginal,
  - valorActual:= valorActual * (multiploActual + 1),
  - multiploActual:= multiploActual + 1.
- Es importante notar que el tamaño del heap nunca supera n, ya que en cada iteración en la cual se agrega un elemento, previamente se extrae uno. Como el Heap nunca supera n elementos, la complejidad de las operaciones de inserción y extracción es $O(\log n)$.
- Luego devolvemos el arreglo
- **Complejidad**: $O(nk \log n)$.

Pseudocódigo:

```
proc ordenarMultiplos(A: array<int>, k: int):array<int>
    vectorAux := new vector<tuple<int, int, int>>

    for elem in A: // O(n)
        vectorAux.agregarAtras(<elem, elem, 1>) // O(1) amortizado
    endfor

    // ordenada por segundo componente de la tupla
    minHeap := colaDePrioridadDesdeSecuencia(vectorAux)

    res := new Array<int>(A.length * k)
    indiceRes := 0

    while(not minHeap.colaDePrioridadVacia()): //O(nklog(n))
        tupla := minHeap.desencolarMin() //O(log(n))
        res[indiceRes] := tupla[1]
        indiceRes++

        if(tupla[2] != k):
            minHeap.encolar(<tupla[0], tupla[1]*(tupla[2]+1), tupla[2]+1>)//O(log(n))

    endwhile

    return res
endproc
```

Ejercicio 15. Se tiene un arreglo A de n números naturales. Sea $m := \text{máx}\{A[i] : 1 \leq i \leq n\}$ el máximo del arreglo. Se desea dar un algoritmo que ordene el arreglo en $O(n \log m)$, utilizando únicamente arreglos y variables ordinarias (i.e., sin utilizar listas enlazadas, árboles u otras estructuras con punteros).

a) Implementar la función

$$\texttt{proc raroSort}(\text{in } A : \text{Array}\langle\mathbb{N}\rangle) : \text{Array}\langle\mathbb{N}\rangle$$

que resuelve el problema planteado. La función debe ser de tiempo $O(n \log m)$, dónde n es el tamaño del arreglo y $m = \text{máx}\{A[i] : 1 \leq i \leq n\}$.

b) Calcular y justificar la complejidad del algoritmo propuesto.

Solución

Idea:

- Recorrer el arreglo y convertir cada número a su correspondiente en base 2 (es decir, a bits) O(n*log(m))
- Ordenar los bits usando radixSort desde el bit menos significativo al más significativo O(n*log(m))
- Recorrer el array ordenado y reconvertir cada número a su correspondiente en base 10 O(n*log(m))

Ejercicio 16. *Arreglos de enteros.*

a) Dar un algoritmo que ordene un arreglo de n enteros positivos menores que n en tiempo $O(n)$.

b) Dar un algoritmo que ordene un arreglo de n enteros positivos menores que n^2 en tiempo $O(n)$. *Pista: Usar varias llamadas al ítem anterior.*

c) Dar un algoritmo que ordene un arreglo de n enteros positivos menores que n^k para una constante arbitraria pero fija k.

d) ¿Qué complejidad se obtiene si se generaliza la idea para arreglos de enteros no acotados?

Solución

Idea general del ejercicio:

a) Ordenar con countingSort

b) Reescribir cada número del arreglo en base n (Me quedo con la $tupla < elem\ div\ n, elem\ mod\ n >$), ordenar con radixSort y volver a base 10

c) Reescribir cada número del arreglo en base n (Da como resultado una tupla de k dígitos), ordenar con radixSort y volver a base 10

d) O(n*log(maximo)), ya que para reescribir el maximo en base n se necesitan $log_n(maximo)$ dígitos

Ejercicio 17. Se nos pide ayudar a un herborista que quiere poder organizar sus ingredientes para determinar qué hierbas le conviene recolectar. Para ello cuenta con su propio inventario. Como no es una persona muy organizada, puede tener distintas hierbas del mismo tipo en distintas alacenas o cofres. Luego de realizar una inspección de su lugar de trabajo, nos entrega una secuencia de n tuplas que constan de una hierba, identificada por su nombre, y la cantidad que se encontró. El nombre de cada hierba tiene como máximo 100 caracteres, de acuerdo al estándar de la Organización Mundial de Herboristas. El herborista cuenta a su vez con su libro de creaciones, que le permite saber en cuántas recetas se utiliza cada hierba.

Se necesita saber cuáles son las hierbas que se usan en más creaciones y, en caso de empate, deberían aparecer primero aquellos de las que tiene menos reservas. La complejidad esperada en el *peor caso* es de $O(n + h\ log(h))$, donde h es la cantidad de hierbas distintas con las que cuenta el herborista.

```
proc Recolectar(in s:Vector<tupla<string,int>>, in u:Diccionario<string,int>):Vector<string>
```

Ejemplo:

```
stock = [ ("Diente de León", 10), ("Menta", 4), ("Margarita", 13),
    ("Lavanda", 12), ("Diente de León", 5), ("Margarita", 6) ]

usos = {"Diente de León": 5, "Menta": 1, "Margarita": 3, "Lavanda": 5}

Recolectar(stock, usos) = ["Lavanda", "Diente de León", "Margarita", "Menta"]
```

Los primeros son la lavanda y el diente de león porque ambos tiene 5 usos, pero aparece primera la lavanda porque hay menos stock. En tercer lugar tenemos la margarita que tiene 3 usos. Finalmente, en último lugar está la menta, que tiene un solo uso.

a) Se pide escribir el algoritmo de `Recolectar`. Justificar **detalladamente** la complejidad y escribir todas las suposiciones sobre las implementaciones de las estructuras usadas, entre otras.

b) ¿Cuál sería el *mejor caso* para este problema? ¿Cuál sería la cota de complejidad más ajustada?

Solución

- **¿Qué asumimos?**
  - Que el diccionario u pasado como parámetro está implementado sobre Trie
  - Eso lo asumimos así porque, al estar acotado el nombre de cada hierba, consultar su valor es O(1)

a) **Juntamos los datos de stock en un vector auxiliar stockTotal**

- Ordenamos el vector s con radixSort según primer componente de la tupla para que estén juntas y sea fácil juntarlas
- Recorremos el vector s ordenado y metiendo la información en nuestro vector auxiliar stockTotal aprovechando que están juntas para contar su stock total
- Observar que nuestro vector auxiliar tendrá tamaño h

- **Complejidad**: $O(n)$.

b) **Juntamos los la información de stockTotal con los usos**

- Creamos un vector auxiliar stockTotalYUsos de $tuplas < nombreHierba, stockTotal, usos >$
- Recorremos el vector stockTotal y vamos consultando sus usos en el diccionario u y agregando dicha información al vector auxiliar stockTotalYUsos
- **Complejidad**: $O(h)$.

c) **Ordenamos el vector stockTotalYUsos**

- Ordenamos por segundo componente de la tupla en orden ascendente en O(h*log(h))
- Ordenamos de forma estable con mergeSort por tercer componente de la tupla en orden descendente
- **Complejidad**: $O(h * log(h))$.

d) **Armo el vector solución**

- Recorro el vector stockTotalYUsos ordenado y agrego en mi vector solución el primer componente de la tupla
- **Complejidad**: $O(h)$.

Complejidad Total: $O(n + h * log(h))$.

Pseudocódigo:

```
proc Recolectar(s: Vector<tupla<string, int>>, u: DiccionarioDigital<string, int>): vector<string>

    radixSort(s, 100) // O(n*100) = O(n)

    stockTotal := new Vector<tupla<string, int>>

    stockTotal.agregarAtras(s[0])
    indiceSTotal := 0

    for(int i := 1; i< s.longitud(); i++): // O(n)
        if(s[i][0] != stockTotal[indiceSTotal][0]):
            stockTotal.agregarAtras(s[i]) // O(1) amortizado
            indiceSTotal++
        else:
            stockTotal[indiceSTotal][1] := stockTotal[indiceSTotal][1] + s[i][1]
    endfor

    stockTotalYUsos := new vector<tupla<string, int, int>>

    for elem in stockTotal: // O(h)
        hierba := elem[0]
        stockHierba := elem[1]
        usosDeHierba := u.obtener(hierba) // O(100) = O(1)
        stockTotalYUsos.agregarAtras(<hierba, stockHierba, usosHierba>)
    endfor

    mergeSort(stockTotalYUsos, ascendentePorSegundaComponente) //O(h*log(h))

    mergeSort(stockTotalYUsos, descendentePorTercerComponente) //O(h*log(h))

    res := new vector<string>

    for elem in stockTotalYUsos: // O(h)
        res.agregarAtras(elem[0]) // O(1) amortizado
    endfor

    return res
endproc
```

b) El mejor caso sería que solo haya un solo tipo de hierba, quedando la complejidad en O(n) (Notar que no estamos fijando el tamaño de la entrada porque, de igual forma, s crece)

Ejercicio 18. Dado un arreglo de pares `<estudiante, nota>`, queremos devolver un arreglo de pares `<estudiante, promedio>`, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta.

Los estudiantes se representan con un número de libreta (puede considerar que es un número de a lo sumo 10 dígitos), y la nota es un número entre 1 y 100. La cantidad de estudiantes se considera no acotada.

Se pide dar un algoritmo que resuelva el problema con complejidad $O(n + m \log m)$, donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes.

Ejemplo:

```
Entrada: <12395, 72>, <45615, 81>, <12395, 94>, <45615, 100>, <78920, 81>,
         <12395, 90>, <45615, 71>, <78920, 50>
Salida: <12395, 85.3333>, <45615, 84>, <78920, 65.5>
```

a) Describa cada etapa del algoritmo con sus palabras, y justifique por qué cumple con las complejidades pedidas.

b) Escriba el algoritmo en pseudocódigo. Puede utilizar (sin reescribir) todos los algoritmos y estructuras de datos vistos en clase.

Solución

Resuelto en el campus clase Sorting

Ejercicio 19. Se está por realizar un gran censo de perros en la Ciudad de Buenos Aires. La información recaudada llegará en forma de un arreglo de tuplas `<Nombre, Raza, Edad, Peso>`. Las razas son `strings` de largo acotado L, pero no sabemos cuántas razas pueden existir, aunque sí sabemos que entre ellas está la mejor raza de todas: raza Perro. Las edades serán enteros positivos entre 1 y 20 y los pesos serán de tipo `float`.

Proponga un algoritmo que organice la información en dos arreglos:

- Uno con los perros raza Perro ordenados por edad y, en caso de empate, por peso, ambos en forma ascendente
- Otro arreglo con la información de las razas y la cantidad de perros de cada una en forma de tuplas `<Raza, int>`, ordenadas en forma decreciente por la cantidad y, en caso de empate, alfabéticamente por la raza

La complejidad temporal del algoritmo debe ser $O(n + p.log(p))$ siendo n es el total de perros censados y p el total de perros raza Perro.

Ejemplo: Si recibimos

```
[ <Nulus, Perro, 4, 15.3>, <Joe, Cocker, 14, 8.0>, <Lady, Caniche, 10, 6.1>, <Dobby, Perro, 7, 18.3>
    <Res, Dogo, 3, 50.0>, <Haru, Perro, 7, 30.0>, <Clementina, Perro, 14, 10.0>, <Choqui, Caniche, 16, 5.0> ]
```

Deberíamos devolver:

```
P: [ <Nulus, 4, 15.3>, <Dobby, 7, 18.3>, <Haru, 7, 30.0>, <Clementina, 14, 10.0> ]
R: [ <Caniche, 2>, <Cocker, 1>, <Dogo, 1> ]
```

Solución

resuelto en el campus clase Sorting

Ejercicio 20. Se tienen dos arreglos de enteros, el primero contiene los elementos a ordenar en la salida y el segundo determina un criterio de ordenamiento.

`proc OrdenarSegunCriterio(in` s `: Array<int>, in` $crit$ `: Array<int>) : Array<int>`

El usuario quiere reordenar s teniendo en cuenta el orden de aparición de los elementos definidos en $crit$. Los elementos que están presentes en s, pero no están presentes en $crit$, deben encontrarse al final del arreglo res y estar en orden creciente. El arreglo $crit$ es de tamaño menor o igual que s, no tiene repetidos y puede contener elementos extra que no son parte de s.

A continuación se presenta un ejemplo:

```
s    = [5,9,3,7,3,4,3,5,1,8,4]
crit = [3,5,2,7,6]

OrdenarSegunCriterio(s, crit) = [3,3,3,5,5,7,1,4,4,8,9]
```

Como en *crit* encontramos primero a 3, luego 5 y detrás 7, todas las apariciones de 3 aparecen primeras, seguidas por las apariciones de 5 y finalmente las de 7. Como 2 y 6 no están en s, son ignorados. Luego, se encuentran todos los elementos de s que no sean 3, 5 o 7 en orden ascendente.

Se pide dar un algoritmo que haga lo requerido con complejidad de $O(n.log(n))$ siendo n la cantidad de elementos de s.

Solución

Algoritmo:

- Copio el arreglo crit a otro arreglo llamado critYPosicion el cual tiene tuplas con primer componente el elemento y segundo su posición en crit
- Ordeno el arreglo critYPosicion en orden ascendente por primer componente
- Por cada elemento de s, hago búsqueda binaria sobre critYPosicion según primer componente para separar la información en dos vectores dependiendo de si está o no está: elemYPosicionEnCrit: $vector < tuple < int, int >>$ y noEstaEnCrit: $vector < int >$
- Observar que en elemYPosicionEnCrit de paso guardé la información de la posición donde estaba originalmente en crit
- noEstaEnCrit lo ordeno en forma creciente con mergeSort
- Para ordenar elemYPosicionEnCrit, me creo minHeap a partir del mismo que ordene según segunda componente (O sea, según la posición que tenía en crit)
- Voy desencolando ese heap y me guardo la información en un vector: estaEnCrit
- Concateno estaEnCrit y noEstaEnCrit, lo convierto en array y lo devuelvo

Pseudocódigo:

```
proc OrdenarSegunCriterio(s: array<int>, crit: array<int>): array<int>

    critYPosicion := new array<int>(crit.length)
    for(int i = 0; i< crit.length; i++): //O(n)
        critYPosicion[i] := <crit[i], i>
    endfor

    elemYPosicion := new Vector<tuple<int, int>>
    noEstaEnCrit := new Vector<int>

    //Asumo que tengo la función búsqueda binaria que me da -1 si no está
    //Y si está, me da el índice donde está
    for elem in s: // O(n * log(n))
        indiceBBinaria := busquedaBinaria(elem, critYPosicion) //O(log(n))

        if(indiceBBinaria == -1):
            noEstaEnCrit.agregarAtras(elem) // O(1) amortizado
        else:
            elemYPosicion.agregarAtras(critYPosicion[indiceBBinaria])
    endfor

    mergeSort(noEstaEnCrit, menorAMayor) // O(n*log(n))

    minHeap := colaDePrioridadDesdeSecuencia(elemYPosicion) //O(n)
    //El cual ordena según segundo elemento de la tupla

    estaEnCrit := new Vector<int>
```

```
        while(not minHeap.colaDePrioridadVacia()) //O(n*log(n))
            elem := minHeap.desencolarMin() //O(log(n))
            estaEnCrit.agregarAtras(elem[0]) //O(1) amortizado
        endwhile

        resVector := estaEnCrit.concat(noEstaEnCrit) //O(n)

        res := vectorToArray(resVector) //O(n)

        return res

    endproc
```